MERCY LEDGER

Technical and Implementation Report

Plain-language guide to the architecture, algorithms, interfaces, validation, and delivery status

Prepared from the project implementation pack | 23 October 2026

1. Executive summary

Mercy Ledger is a Python-based decision-support prototype for investigating officers, bank nodal officers, and authorized reviewers handling cyber-fraud fund reconciliation.

It takes a structured account statement, complaint records, account details, dates, and optional invoice information. It then produces an auditable analysis of disputed money, remaining account balances, deadlines, possible review actions, and draft documents.

The system is not a bank integration, criminal-fraud detector, legal decision-maker, or replacement for NCRP/CFCFRMS. It calculates and explains what the supplied records support. A human officer remains responsible for every decision.

Current status: the attribution engine is implemented and passes its checker. Six backend modules and the frontend are intentionally still placeholders. Therefore the full application is not complete yet.

2. What problem the system solves

A cyber-fraud complaint may identify money that entered an account, but the account may also contain legitimate funds, other victims' money, or money that has already left through later debits.

Manual spreadsheet analysis can be difficult to reproduce. It can also accidentally freeze an entire account, count the same principal twice, or confuse a transaction relationship with proof of guilt.

Mercy Ledger addresses this by using a fixed data contract, deterministic decimal arithmetic, chronological transaction replay, explicit assumptions, and machine-checkable golden outputs.

The tool answers questions such as: Which disputed credits can be matched? How much remains in the account under the demo model? Which outgoing debits consumed disputed funds? What deadlines are relevant? What cash impact would different recommendations have?

3. High-level architecture

The project is divided into a baseline application and three implementation priorities.

Baseline: shared rules, data contract, presets, golden fixtures, attribution engine, API glue, and acceptance checker.

Priority 1: complete the core MVP. Implement the simulator, triage, SLA, impact, grievance, orders, and frontend modules.

Priority 2: add judge-visible depth such as conservation invariants, stress testing, review flags, victim explanations, replay controls, and scenario cards.

Priority 3: optional stretch work such as controlled transaction-link graphs, assumption sensitivity analysis, and report fingerprints.

The integration flow is: case input -> attribution -> grievance -> triage -> SLA -> impact -> orders -> API response -> browser UI.

4. Repository map

START_HERE.md: team workflow, scope, known limitations, and release checklist.

INTEGRATION_PROTOCOL.md: ownership rules, merge order, and evidence required before acceptance.

ROLE_AND_PRIORITY_MAP.md: maps each file to the responsible teammate and priority.

BASELINE/mercy_ledger/01_DATA_CONTRACT.md: frozen input and output interfaces.

BASELINE/mercy_ledger/00_RULES_FOR_EVERY_AI.md: engineering, safety, determinism, and handoff rules.

BASELINE/mercy_ledger/presets.py: three built-in demonstration cases.

BASELINE/mercy_ledger/golden/: expected JSON results used by the checker.

BASELINE/mercy_ledger/check_my_work.py: module and whole-pipeline acceptance referee.

BASELINE/mercy_ledger/app.py: FastAPI HTTP layer.

BASELINE/mercy_ledger/pipeline.py: orchestration layer.

BASELINE/mercy_ledger/static/index.html: frontend, currently a placeholder.

5. Input data contract

A case contains case_id, tier, title, description, demo_today, account, opening_balance, transactions, complaints, seizure_timestamp, sla_context, and invoices.

An account contains account_id, account_type, layer, and holder_name. Account types include SAVINGS, CURRENT, NODAL, POOL, and ESCROW.

Each transaction contains txn_id, timestamp, type, amount, utr, and narration. Type is CREDIT or DEBIT.

Each complaint contains complaint_id, victim_label, state, disputed_utr, disputed_amount, and complaint_timestamp.

A complaint is treated as disputed only up to the matching credit amount. The current schema does not identify the receiving account of an outgoing transfer.

Money is represented as strings with exactly two decimal places, for example "150000.00". Timestamps and dates are ISO text.

The opening balance is assumed clean by this demonstration model. That is an analytical assumption, not proof of provenance.

6. Attribution engine: implemented core

File: attribution.py. Public function: analyze_attribution(case)

Step 2 - Sort transactions chronologically using timestamp plus original row index. The row index makes equal timestamps deterministic.

Step 3 - Maintain two balances: clean balance and tainted/disputed pools. Credits matched to complaints add to a victim pool; other credits add to clean funds.

Step 4 - Process debits. Under the explicit demo assumption, disputed funds are consumed before clean funds. This is not presented as a universal statutory algorithm.

Step 5 - Attribute outgoing debits. A debit affecting one victim is DIRECT. A debit affecting multiple victims is divided PRO-RATA according to the victims' remaining amounts.

Step 6 - Record epochs, balances, victim totals, forwarded debit references, retention ratios, holds, warnings, and assumptions.

All monetary calculations use Decimal. Values are rounded half-up to two decimal places only when written into output JSON.

7. Attribution outputs

The attribution result contains case metadata, opening and closing balances, balance_at_seizure, chronological epochs, holds_sequence, victims, clean_ring_fenced, metrics, totals, assumptions, and warnings.

An epoch explains one transaction: event type, amount, UTR, tainted pool, clean balance, ledger balance, and a human-readable note.

A victim result includes disputed amount, attribution method, amount still held, restorable amount, forwarded amount, outgoing debit references, retention ratio, and provisional SOP hold amount.

The output deliberately separates provisional complaint-order holds from attribution-derived restoration values. They may differ and should trigger review rather than being treated as an automatic finding.

8. Triage module: planned Priority 1 work

File: triage.py. Public function: triage(case, attribution, grievance=None).

Its role is to produce a transparent, project-defined scorecard and recommendation. Expected recommendation values are TARGETED_LIEN_ONLY, DIGITAL_SUSPENSION_PENDING_EDD, and SEIZURE_S106.

The scorecard is not an official MHA/I4C score and is not a validated fraud classifier. It is decision support based on supplied evidence and configured thresholds.

The result must include rationale, score, thresholds, metrics, factors, citations, human_signoff_required, and a disclaimer.

The module must never call an account holder guilty or innocent. A review recommendation is not legal authorization.

9. SLA module: planned Priority 1 work

File: sla.py. Public function: compute_sla(case, as_of).

It produces five clocks in a fixed order: BANK_GRIEVANCE_7D, IO_DECISION_15D, INTERIM_CUSTODY_15D, HOLD_REVIEW_90D, and SUSPENSION_REVIEW_45D.

Status values are NOT_STARTED, ON_TRACK, DUE_SOON, OVERDUE, and COMPLETED.

The 45-day suspension review is a proposed project checkpoint, not an SOP clause. The UI must label it exactly that way if displayed.

The date comes from the case demo_today or an explicitly supplied as_of value. No system clock is allowed.

10. Impact module: planned Priority 1 work

Files: impact.py. Public functions: case_impact(case, attribution, triage) and aggregate_impact(items).

Case impact compares a blanket restriction with the selected recommendation. It includes blanket_total_locked, blanket_innocent_locked, mercy_innocent_locked, innocent_capital_freed, victim_recoverable_preserved, and victim_recoverable_in_blanket.

The metric is cash-focused. A digital-services suspension is not treated as cash being frozen; targeted lien or digital-only suspension therefore has zero cash locked in the demo metric.

For a seizure recommendation, the clean balance at the reference timestamp is used. The reference is seizure time when supplied, otherwise the selected closing snapshot.

Aggregate impact combines cases, recommendations, preserved victim funds, reduction percentage, and per-case details.

11. Grievance module: planned Priority 1 work

File: grievance.py. Public function: verify_grievance(case).

It performs structured document-to-ledger consistency checks on invoices. It can compare invoice amount, date, GSTIN format, and claimed UTR against supplied records.

A match means the supplied documents are financially consistent with the supplied ledger. It does not prove that a transaction was legitimate, that a person is innocent, or that the document is genuine.

The result contains case_id, results, disputed_total, disputed_matched, all_disputed_matched, and disclaimer.

GSTIN validation is format-only unless an external registry result is explicitly provided.
No registry lookup is part of this prototype.

12. Draft orders module: planned Priority 1 work

File: orders.py. Public function: build_orders(case, attribution, triage, sla, grievance).

It produces exactly three complete HTML drafts: restoration_order_html, indemnity_bond_html, and edd_packet_html.

Every document must be visibly marked DRAFT, require authorized human review, and state that it is not legally validated or court-ready.

All input-derived values must be HTML-escaped to prevent injection. The module must present amounts already computed by the analysis and must not invent new financial calculations.

The generated documents are workflow aids, not legally operative orders.

stage, and collects errors instead of hiding them.

If an upstream module fails, dependent sections may be null. The errors array reports the stage, exception type, and message so the operator can see incomplete results.

File: app.py. FastAPI exposes preset listing, preset retrieval, case analysis, portfolio impact, health, and the static homepage.

POST /api/analyze-case accepts either preset_id or a complete case object, plus optional as_of.

GET /api/portfolio-impact accepts bounded n_sim and seed values and combines built-in presets with simulator cases.

The API layer uses FastAPI and Pydantic. Pure analysis modules are intended to use only the Python standard library.

14. Determinism and safety rules

Same input must produce the same output. The simulator may use random.Random(seed), but no global randomness or system time is allowed.

No network, AI/LLM calls, machine learning, external data lookup, or live bank integration is part of the product.

Input dictionaries must not be mutated. Modules should work on copies or read-only values.

Money must never be calculated with binary floating point. The checker rejects float values in output.

Untrusted text must be escaped before it is placed into generated HTML.

Errors must be visible. The pipeline uses explicit error collection rather than returning a false success-shaped result.

15. Validation strategy

Golden JSON files are the acceptance targets for the supplied cases. They define exact output keys and expected values.

check_my_work.py checks imports, stub markers, forbidden dependencies, deterministic behavior, output JSON shape, money types, input immutability, and pipeline readiness.

Module checks are run independently: attribution, simulator, triage, sla, impact, grievance, and orders. The all command also checks the complete pipeline.

make_sample_data.py is guarded and refuses to regenerate offline data while required modules are placeholders.

benchmark.py is also guarded. It must not be used to claim performance while the pipeline is still running placeholder code.

16. Current test evidence

Command executed: python check_my_work.py all

PASS: attribution

FAIL : simulator

FAIL: impact

FAIL: grievance

FAIL: orders

FAIL: pipeline (whole system)

Interpretation: attribution is ready; the remaining failures are expected because the six modules contain MERCY_LEDGER_STUB placeholders. The frontend is also still a placeholder page.

17. Priority 1 completion plan

1. Implement simulator.py and verify seeded determinism, valid cases, unique IDs, and non-negative balances.

2. Implement triage.py, sla.py, and impact.py against the data contract and golden fixtures.

3. Implement grievance.py and orders.py with safe escaping and clear draft disclaimers.

4. Replace static/index.html with a UI that loads live API results, shows errors, formats Indian-style amounts, and labels demo/offline data.

5. Run python check_my_work.py all. Do not change the checker or golden fixtures to force a pass.

6. Only after the gate passes, run make_sample_data.py, start uvicorn, and manually test all presets.

7. Add Priority 2 only after Priority 1 is stable and demonstrable.

18. Priority 2 and 3 concepts

Priority 2 should add real conservation checks, replay of attribution epochs, review flags for hold-versus-attribution differences, deterministic victim explanations, and scenario cards.

Conservation checks can verify opening balance + credits - debits = closing balance, pool reconciliation, and allocation bounds. These are software/accounting consistency checks, not fraud accuracy measurements.

Priority 3 can add a controlled multi-account link graph using supplied identifiers, sensitivity analysis for different debit-consumption assumptions, and SHA-256 report fingerprints.

A graph edge proves only that supplied records match under configured criteria. It does not prove control, intent, or exact money provenance after commingling.

Sensitivity outputs are hypothetical comparisons. A fingerprint proves payload equality against a digest, not authorship or legal authenticity.

19. Limitations and responsible interpretation

The initial schema cannot identify a receiving account for an outgoing debit, so it cannot prove a next hop.

The account opening balance is assumed clean in the demo model; real analysis requires adequate statement history.

SOP-derived examples include documented normalization or assumed times where the printed source was incomplete or inconsistent.

The merchant scenario is invented and must not be presented as a verified real-world case.

No output authorizes freezing, seizure, restoration, or prosecution. Authorized officers must review the evidence and applicable procedure.

20. Final conclusion

Mercy Ledger’s strongest completed foundation is its deterministic attribution engine, frozen data contract, golden fixtures, and guarded acceptance checker.

The project is architecturally prepared for an end-to-end demo, but it is not yet a finished application because the supporting backend modules and frontend remain placeholders.

The correct engineering path is to finish and verify Priority 1 before adding advanced features. A trustworthy demo should show both useful calculations and honest limitations.